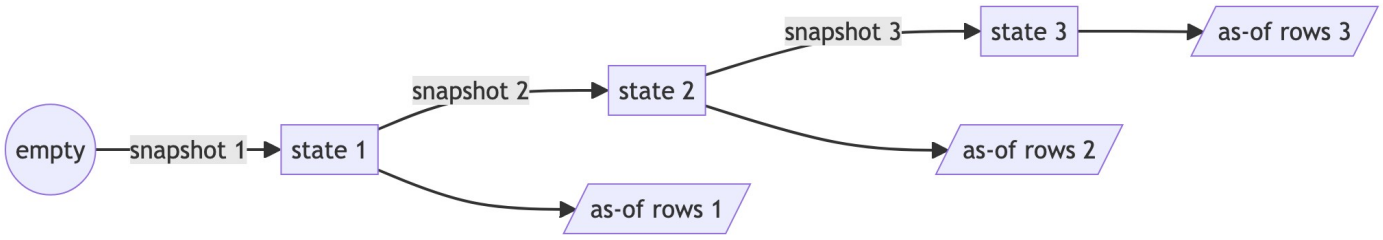
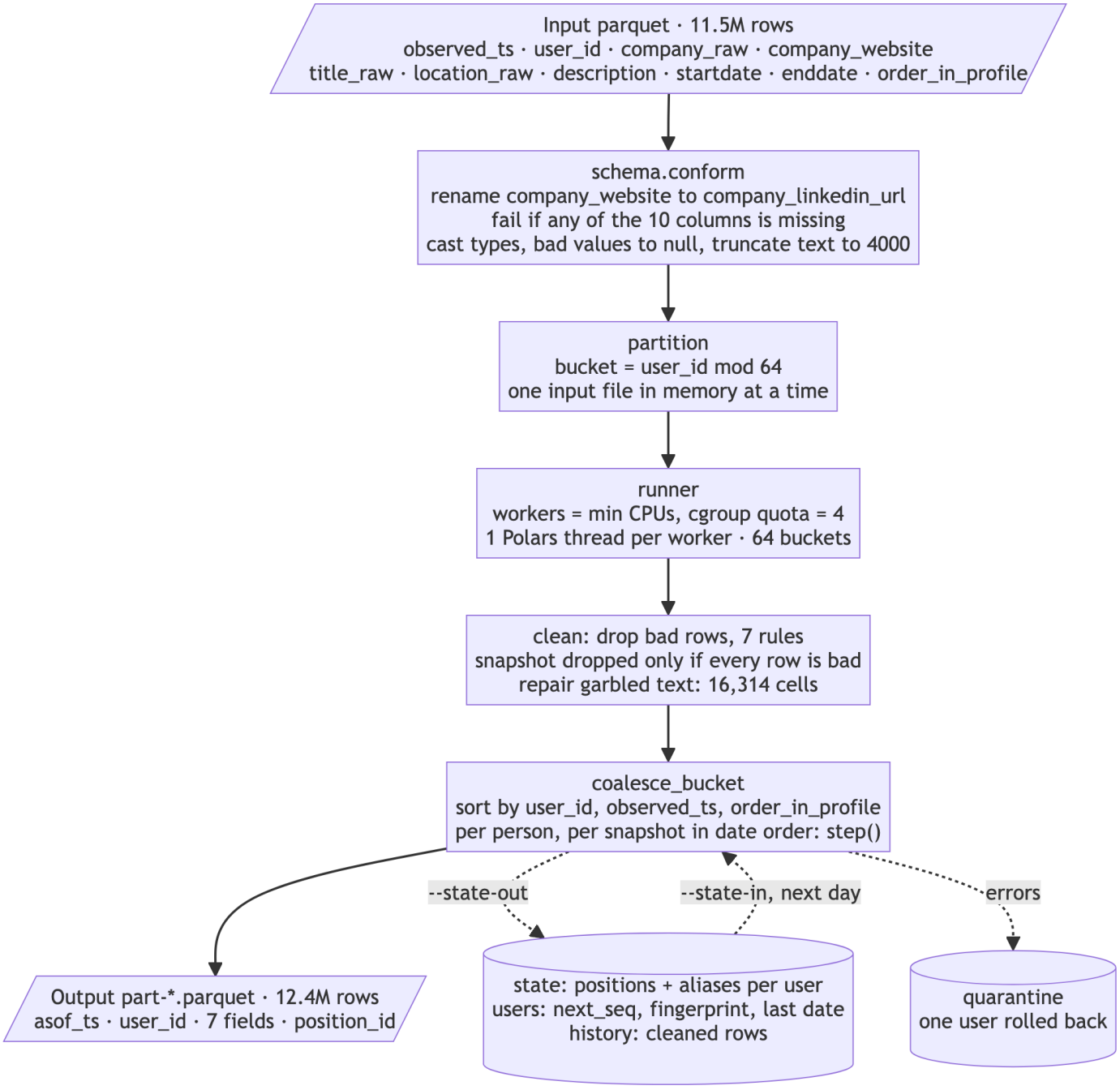
Position Coalescing – Diagrams

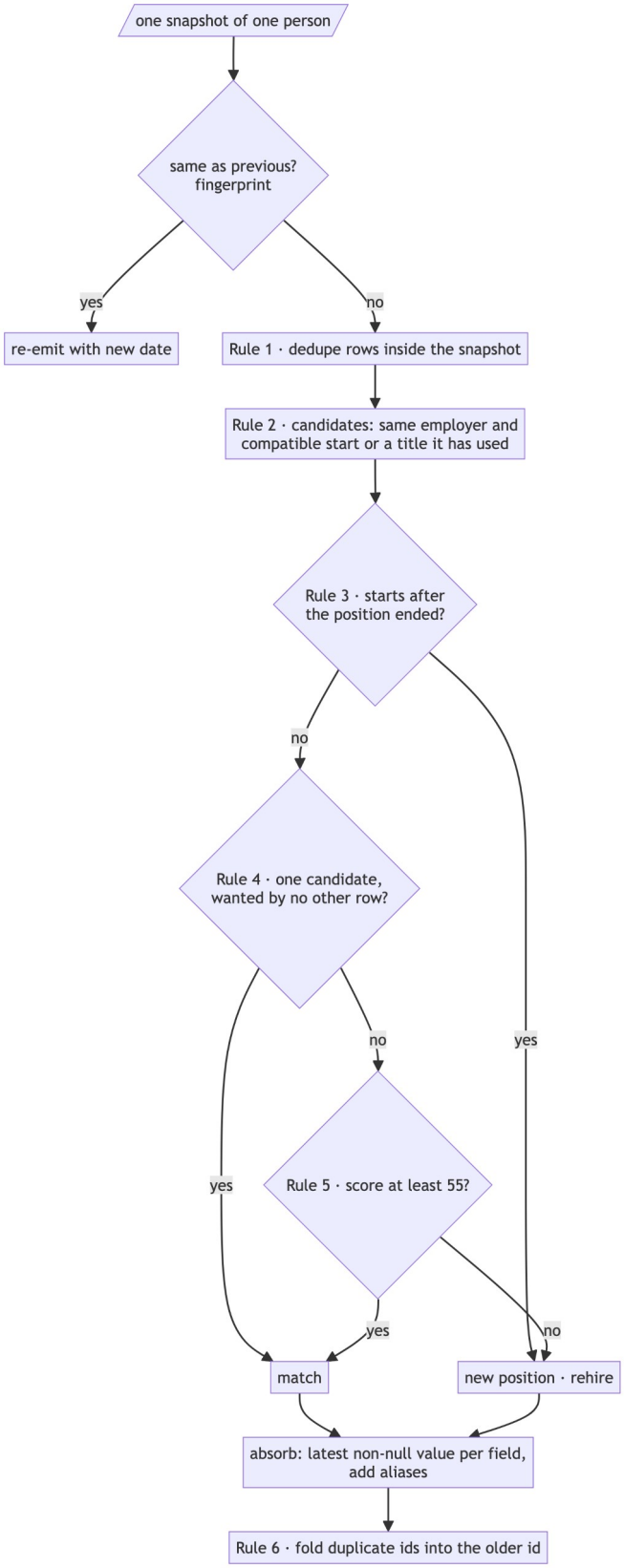
1. One state machine per person



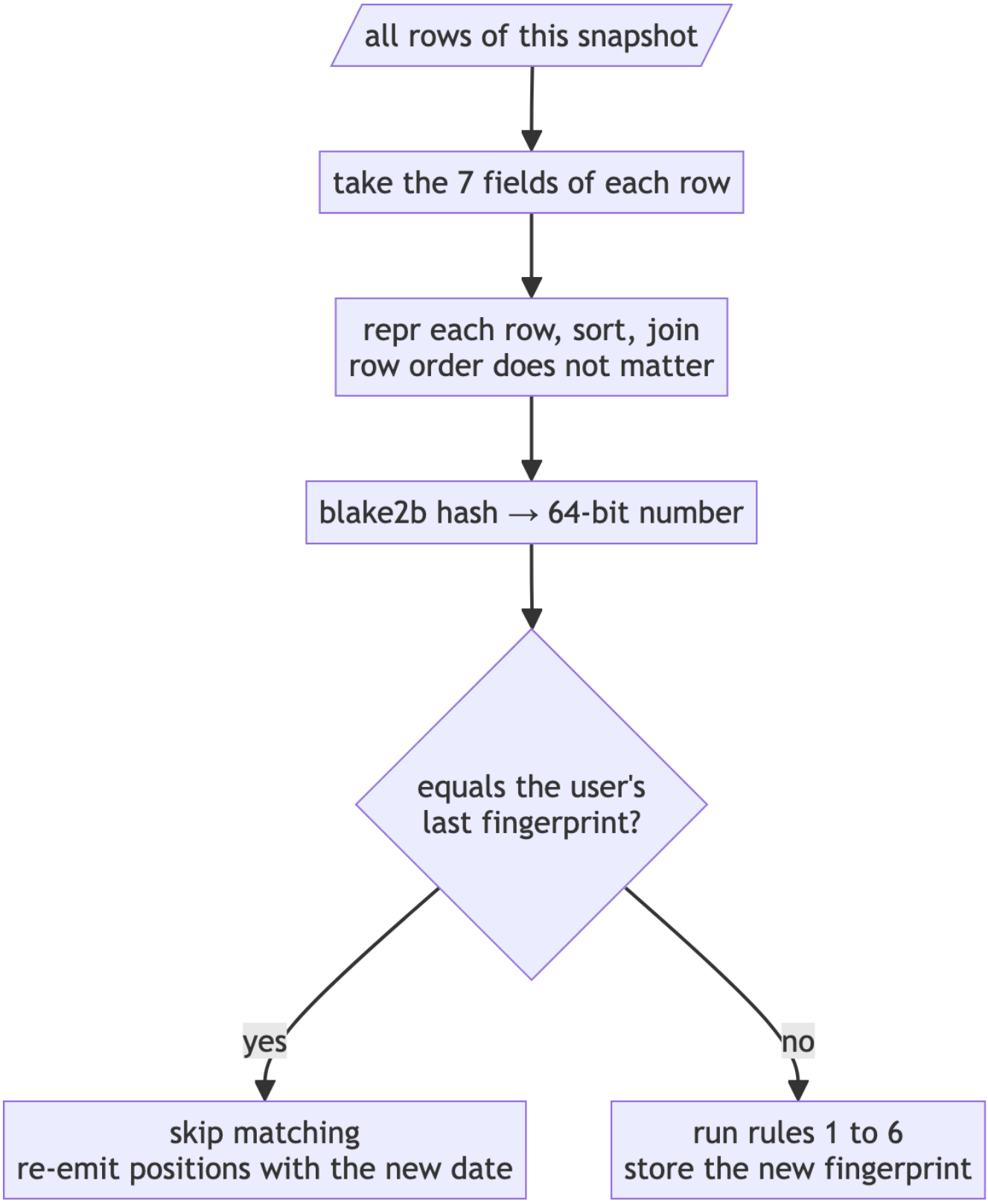
2. Pipeline


Input parquet · 11.5M rows
observed_ts · user_id · company_raw · company_website
title_raw · location_raw · description · startdate · enddate · order_in_profile
schema.conform
rename company_website to company_linkedin_url
fail if any of the 10 columns is missing
cast types, bad values to null, truncate text to 4000
partition
bucket = user_id mod 64
one input file in memory at a time
runner
workers = min CPUs, cgroup quota = 4
1 Polars thread per worker · 64 buckets
clean: drop bad rows, 7 rules
snapshot dropped only if every row is bad
repair garbled text: 16,314 cells
coalesce_bucket
sort by user_id, observed_ts, order_in_profile
per person, per snapshot in date order: step()
--state-out
--state-in, next day
errors
Output part-*.parquet · 12.4M rows
asof_ts · user_id · 7 fields · position_id
state: positions + aliases per user
users: next_seq, fingerprint, last date
history: cleaned rows
quarantine
one user rolled back

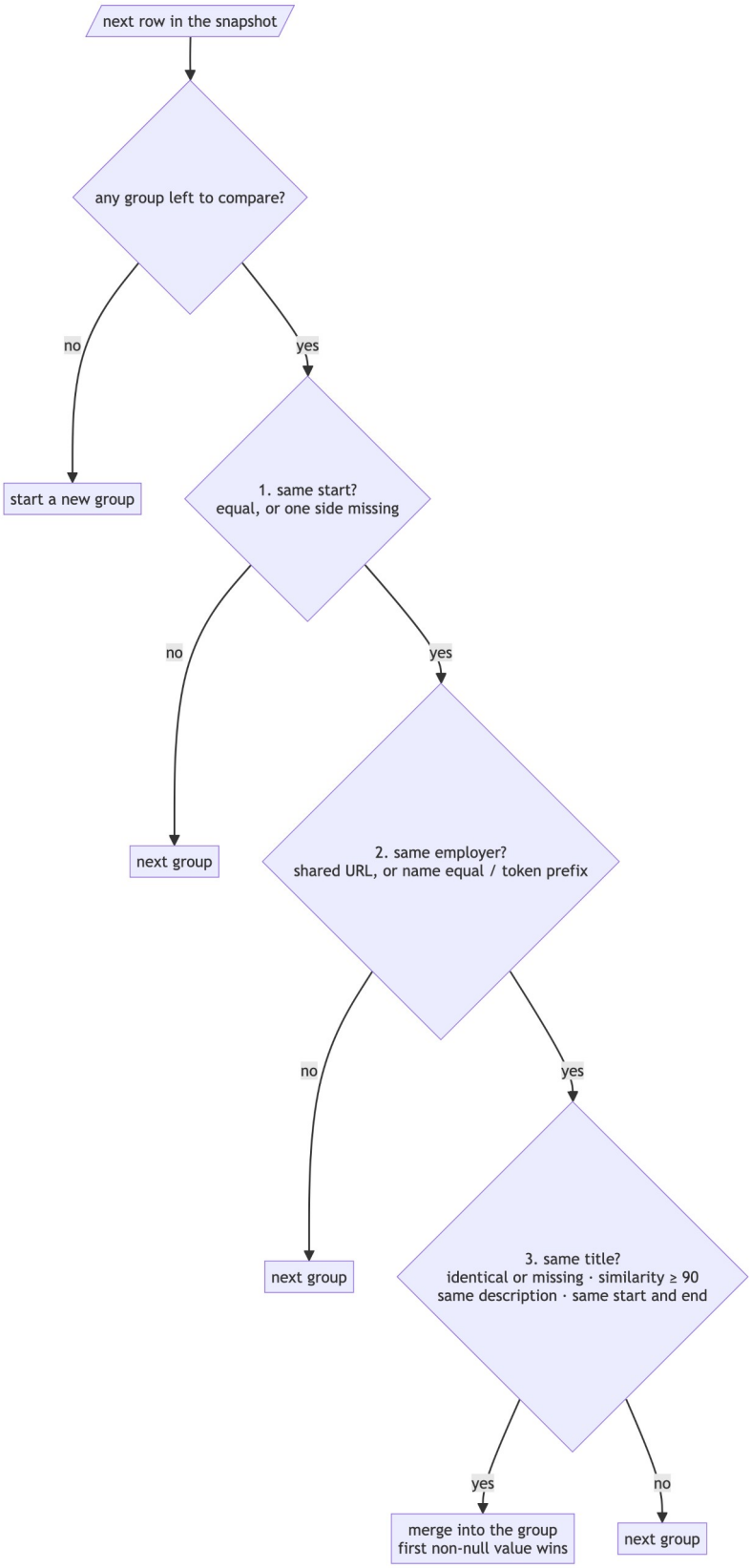
3. Six rules


one snapshot of one person
same as previous? fingerprint
yes
no
re-emit with new date
Rule 1 · dedupe rows inside the snapshot
Rule 2 · candidates: same employer and compatible start or a title it has used
Rule 3 · starts after the position ended?
no
yes
Rule 4 · one candidate, wanted by no other row?
no
yes
Rule 5 · score at least 55?
yes
no
yes
match
new position · rehire
absorb: latest non-null value per field, add aliases
Rule 6 · fold duplicate ids into the older id

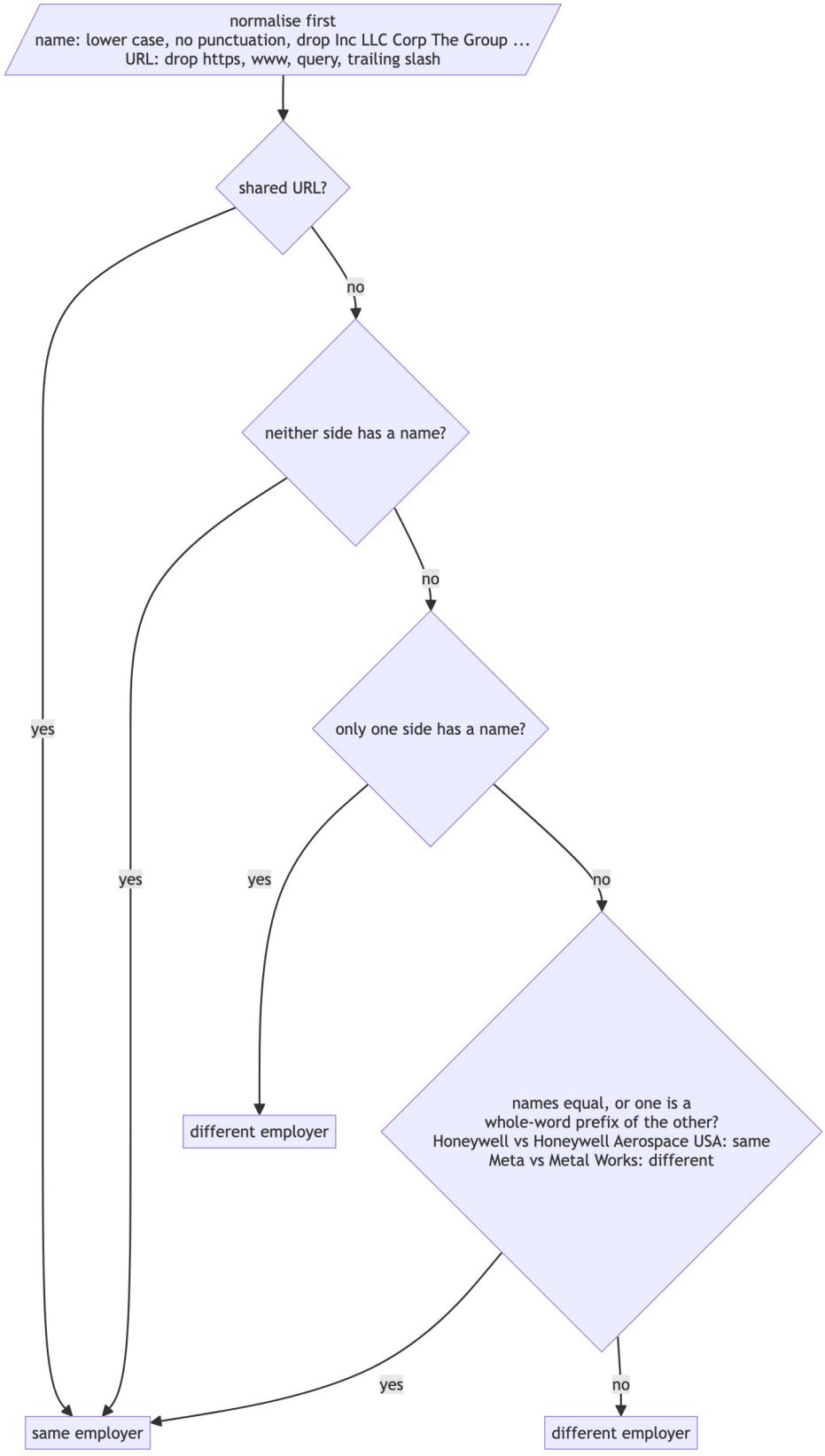
3.0 Fingerprint: same as last time?



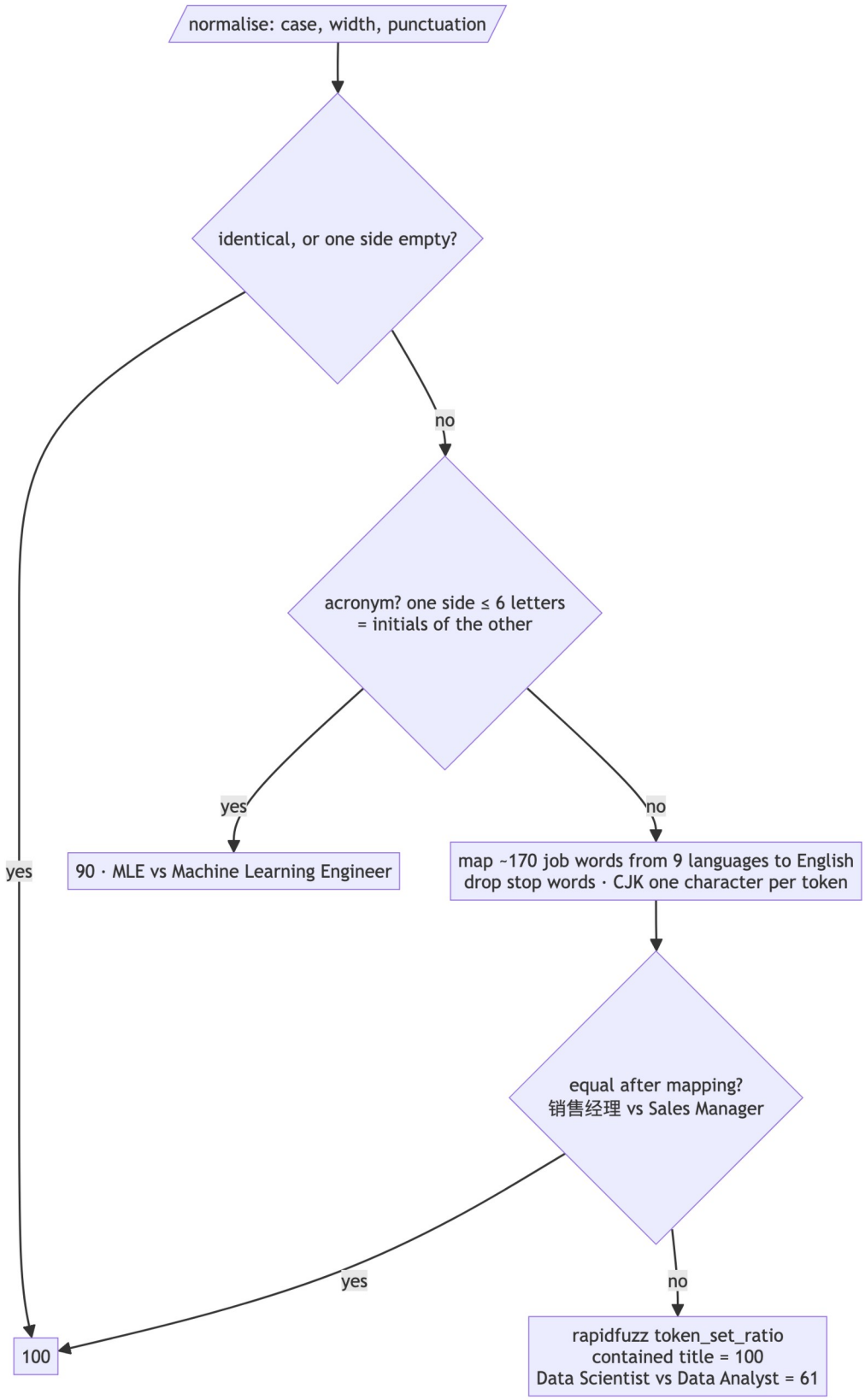
3.1 Rule 1: duplicates inside one snapshot



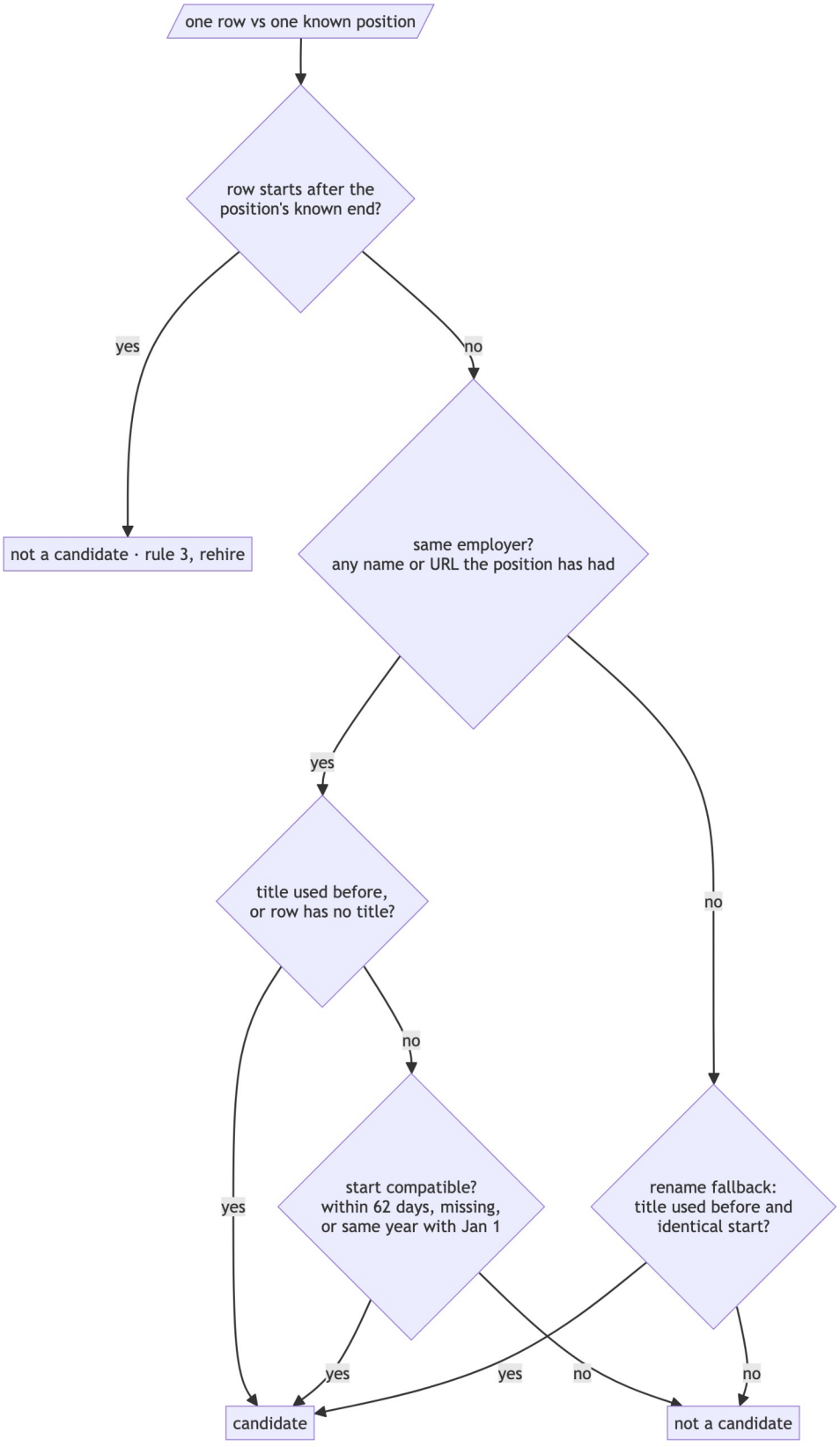
3.1a Same employer?



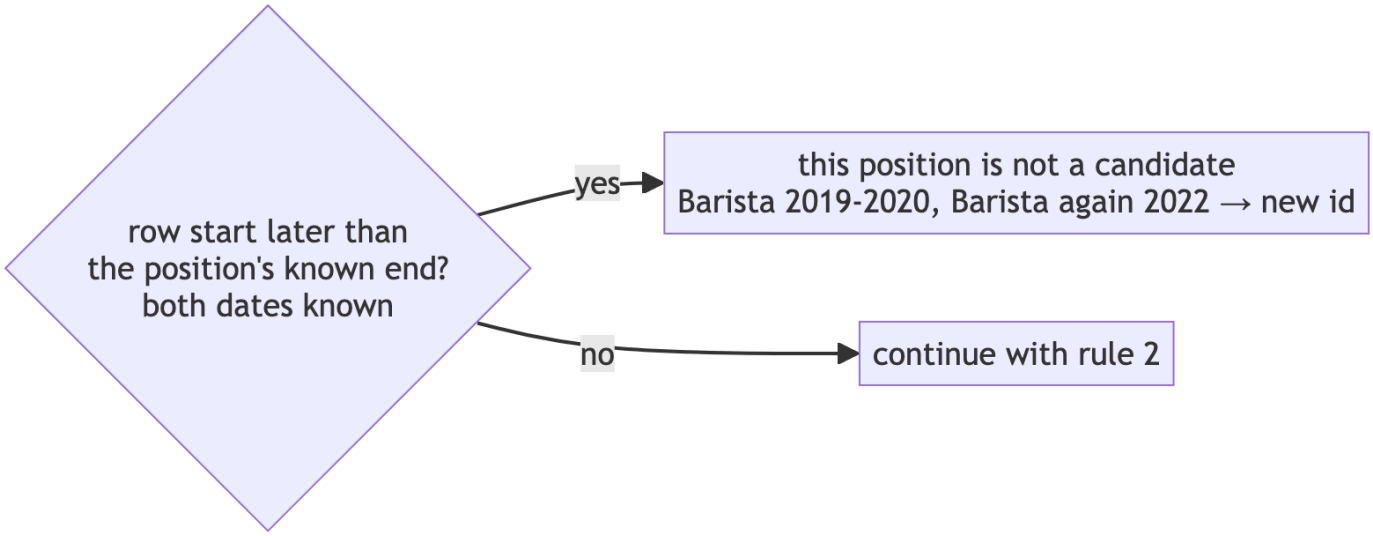
3.1b Title similarity



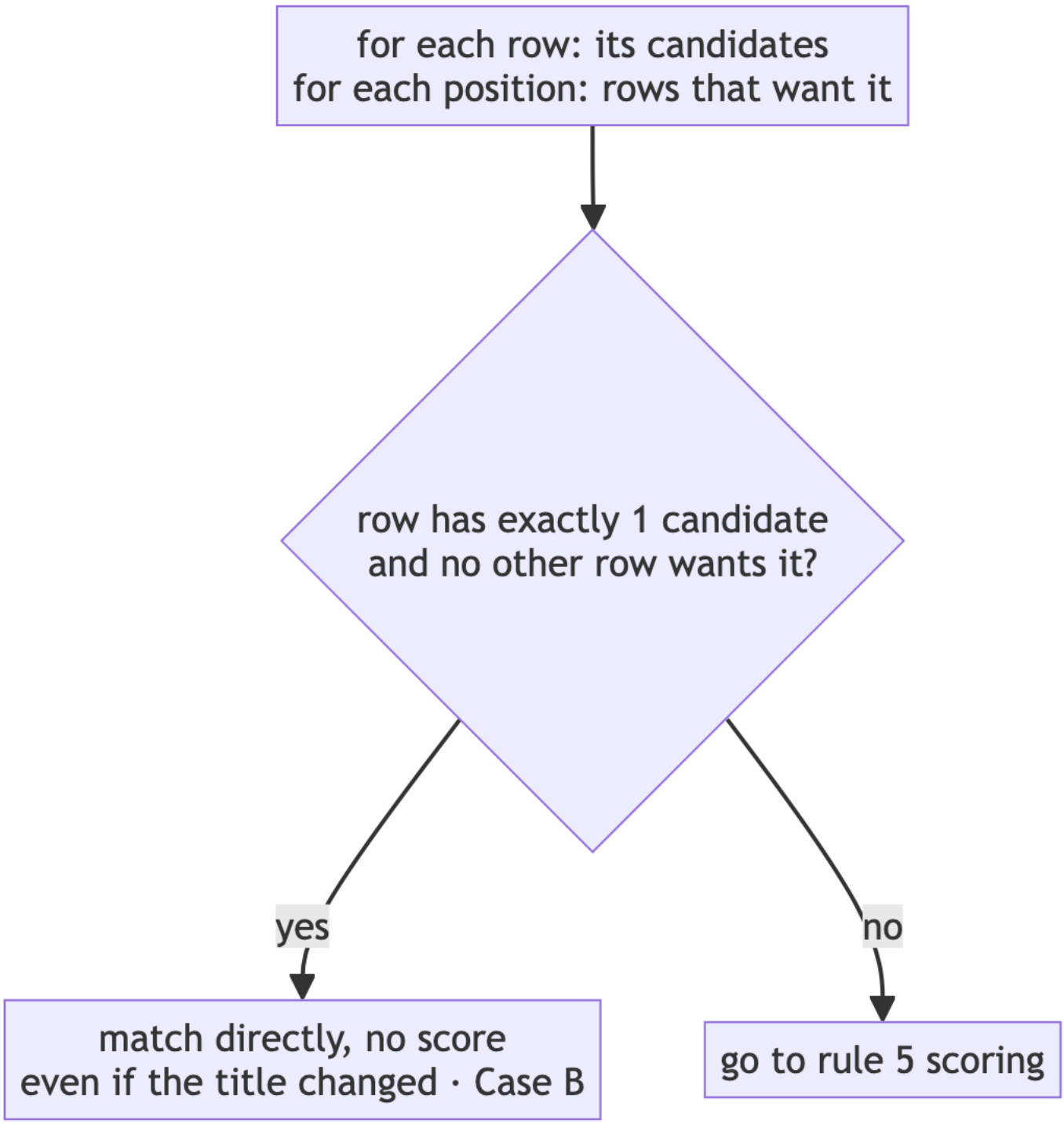
3.2 Rule 2: candidates



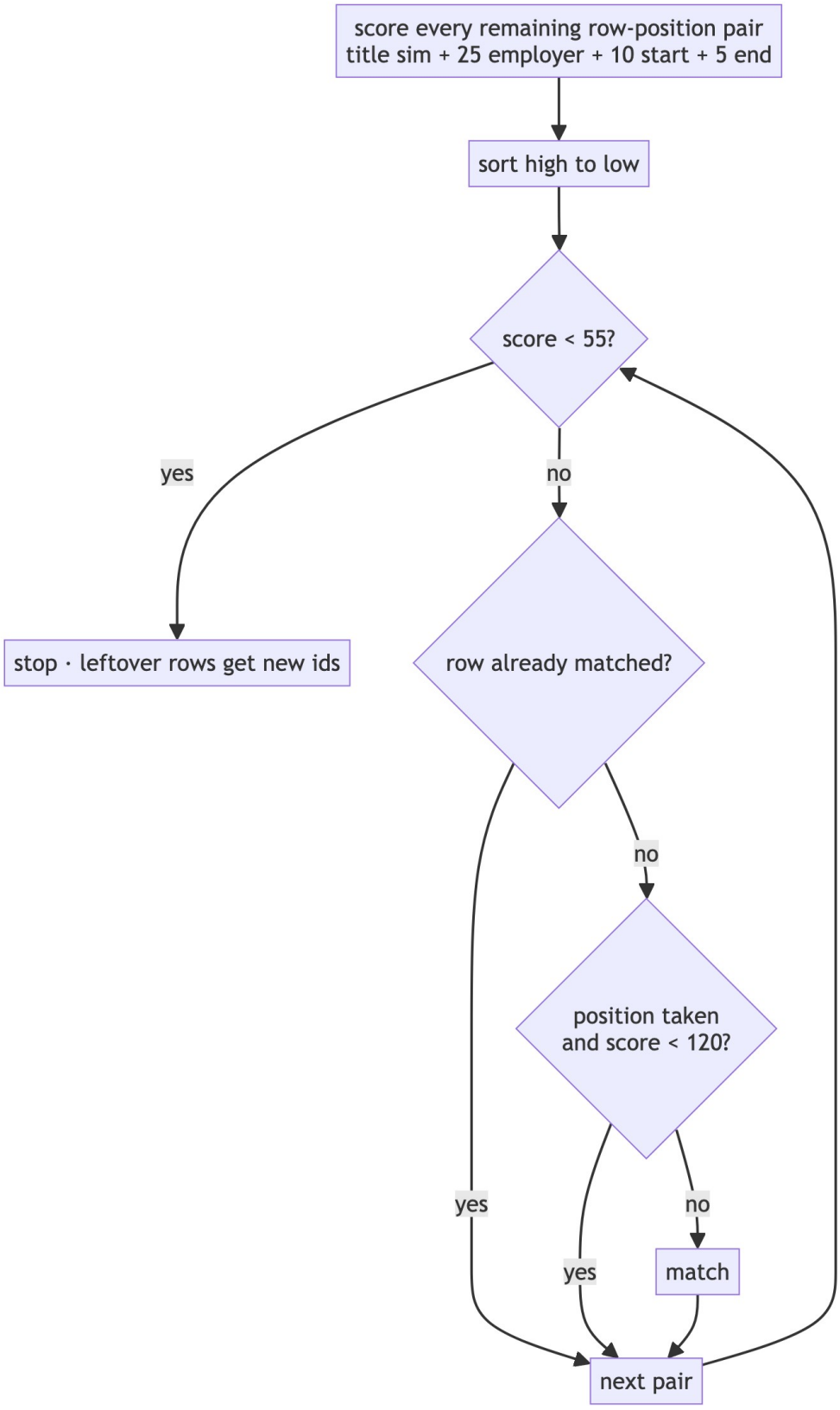
3.3 Rule 3: rehire



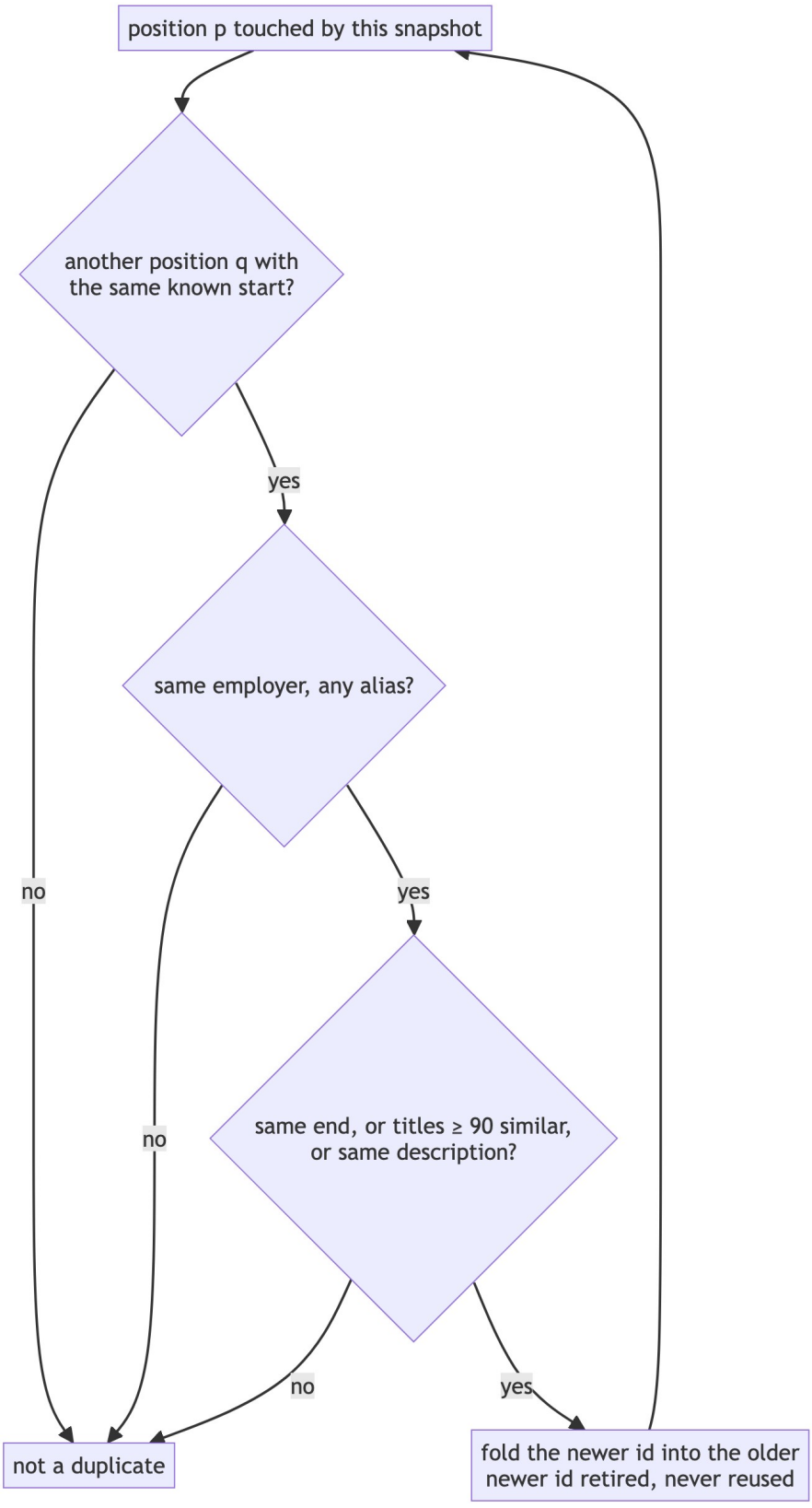
3.4 Rule 4: one uncontested candidate



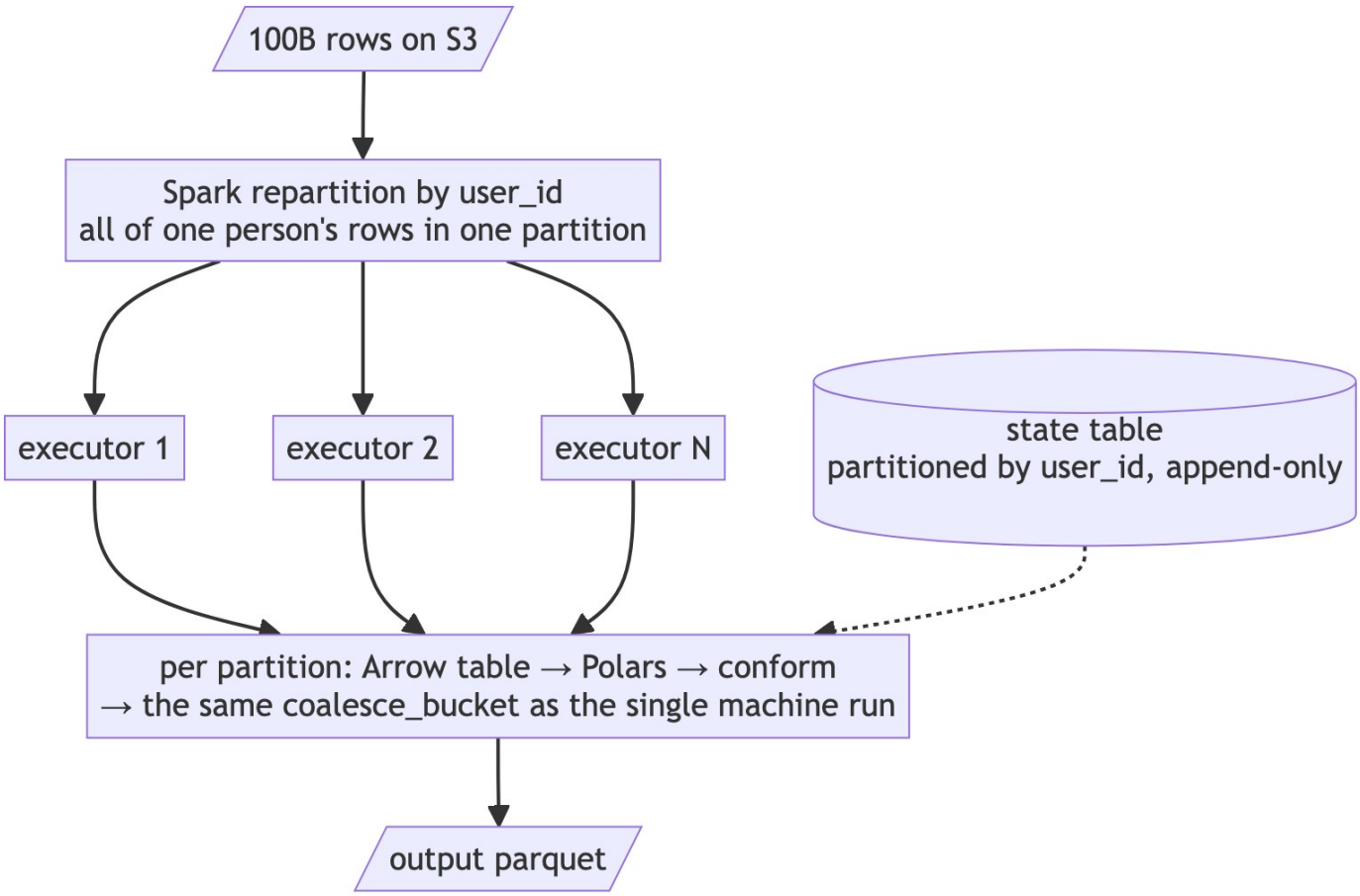
3.5 Rule 5: scoring, greedy



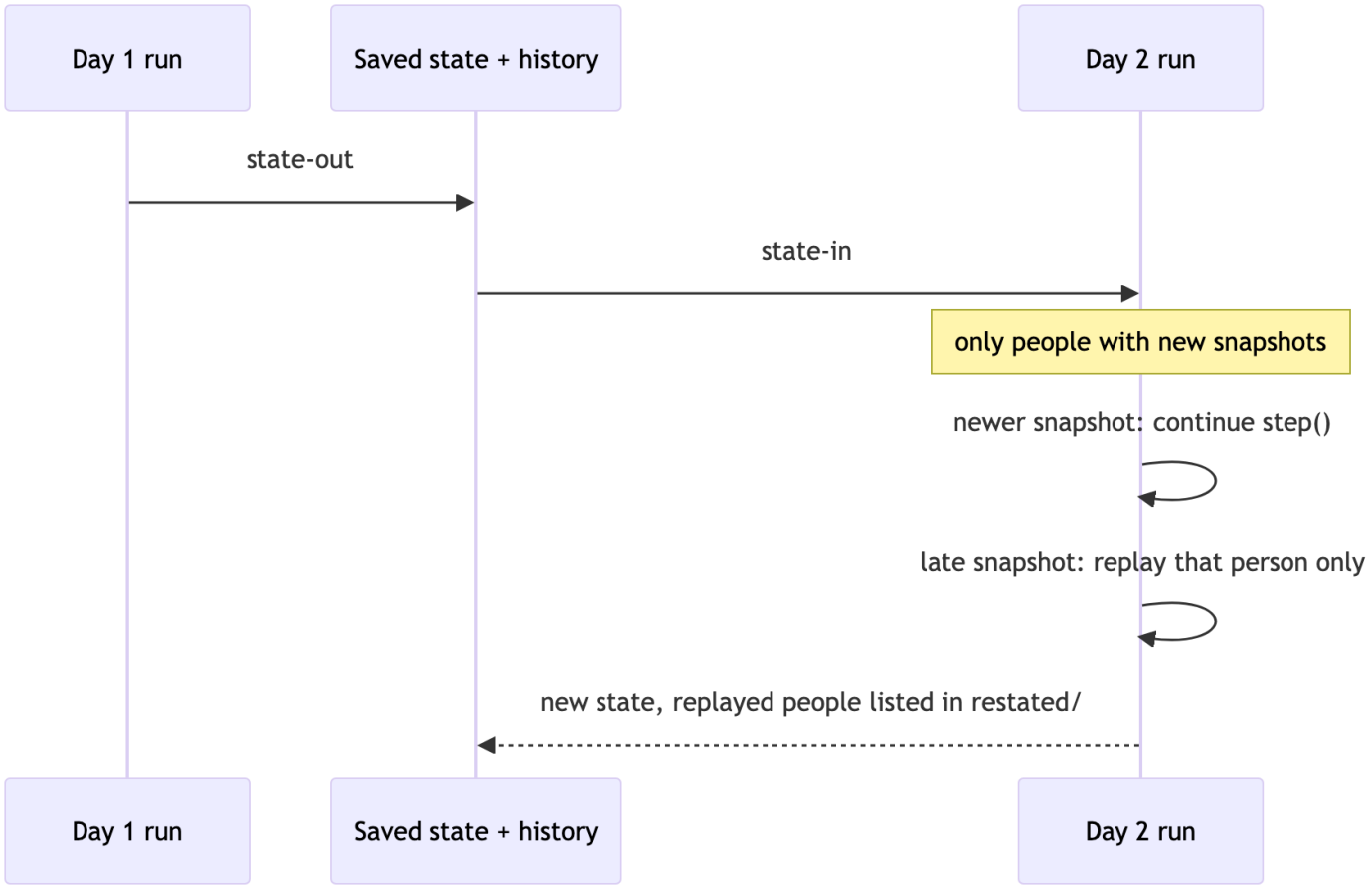
3.6 Rule 6: fold duplicate ids



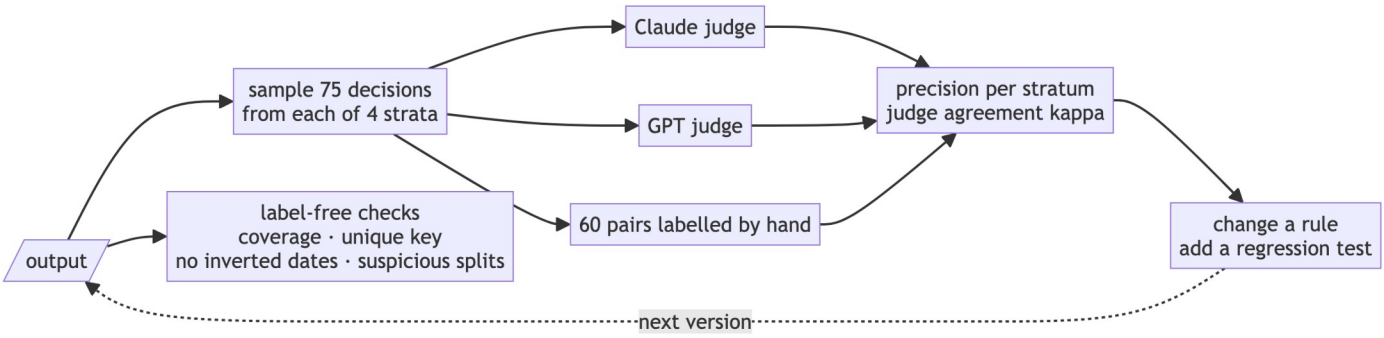
4. Scale to 100B rows



5. Daily updates and late data



6. Evaluation



6a. Two bugs a judge led to

